Dynamic Programming

Topic 2 of 10

12 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Count Perfect Wake Numbers up to N

HARD | asked 2026-05-07 | dp, bit-manipulation

Examples: 196 -> perfect (digits 1,9,6 all distinct, no 0, 9 is not a valley) 23 -> perfect 12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 -- 6 is not a valley because its right neighbor 3 is smaller) 1546 -> not perfect (4 is adjacent to 5 and 6, both strictly larger) 320 -> not perfect (contains 0) 34321 -> not perfect (digit 3 repeats)

```
def is_perfect(n):
```

2. Tree Distance Sum (Re-root DP)

HARD | asked 2026-01-19 | tree, dp, dfs

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Input: undirected tree with N nodes and N-1 edges. Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node. Examples: Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]] ans: [8, 12, 6, 10, 10, 10]

```
def tree_distance_sums(n, edges):
```

3. Bounded Knapsack Variant

MEDIUM | asked 2026-10-16 | greedy, dp, quickselect, sorting

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n). Input: for each item type, its value and the number of copies available. Every item has weight 1. Input: a maximum total weight W. Output: the maximum total value achievable without exceeding W.

```
def max_value_sorted(items, W):
```

4. Longest Non-Decreasing Subarray with One Replacement

MEDIUM | asked 2026-10-04 | array, dp, two-pointer

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Input: an integer array nums. You may change at most one element to any value. Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

```
def longest_with_one_change(a, strict=False):
```

5. Grid Path Count with Diagonal Moves and Checkpoints

MEDIUM | asked 2026-09-29 | dp, grid

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix. Return the total number of distinct paths from start to end. Since every move changes the column, each column can contain at most one visited checkpoint.

```
def count_paths(n, m, checkpoints=(), ordered=F
  alse):
```

6. Minimum-Cost Tree Disconnection

MEDIUM | asked 2026-09-09 | tree, dp

Examples: Negative-weight follow-up: A --(-1)--> B --(-1)--> C Minimum total cut cost: -2 Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

```
def disconnect_cost(n, edges):
```

7. Target Expressions from Ordered Digits

MEDIUM | asked 2026-08-13 | array, backtracking, dp, math

Examples: For digits [2, 3, 4] and target 14, valid outputs include: 2 * (3 + 4) 2 + 3 * 4

```
def expressions_for_target(digits, target):
```

8. Count Word Decompositions into Chemical Element Symbols

MEDIUM | asked 2026-07-14 | string, dp

Examples: Physics has four stated decompositions: P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

```
def count_decompositions(word, symbols=ELEMENTS
  , mod=None):
```

9. Fence Painting Minimum Operations (Gemini Team)

MEDIUM | asked 2026-04-27 | dp, recursion

Examples: n = 5, a = [2, 2, 1, 2, 1] Optimal: 1) Horizontal stroke at h=1 across [1..5] (covers row 1 across whole fence) 2) Horizontal stroke at h=2 across [1..2] 3) Vertical stroke at plank 4 (height 2 still missing row 2) Optimal answer: 3.

```
def paint(a, width=None):
```

10. Jump Game with Profit

MEDIUM | asked n/a | array, dp

Examples: Variant-dependent; the principal recurrence is the one above.

```
def max_profit_fast(arr, profit):
```

11. Count Same-Color Squares in Matrix

MEDIUM | asked n/a | grid, dp

LeetCode 221 variant -- count maximal same-color square sub-grids. DP on each cell as the bottom-right corner. Input: m x n grid of colored cells. Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt). DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1. Sum dp[i][j] across the grid for the count variant; max for the side variant.

```
def count_same_color_squares(g):
```

12. Russian Doll Envelopes

MEDIUM | asked n/a | array, dp, binary-search, sorting

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction. Input: list of (width, height) envelopes. Output: longest chain where each envelope strictly fits in the next. Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights. LIS with patience-sort gives O(n log n). Examples: [[5,4],[6,4],[6,7],[2,3]] -> 3.

```
def max_envelopes(envelopes):
```